



[Course](#)

[Progress](#)

[Dates](#)

[Discussion](#)

Content search 🔍

[🏠](#) [Course](#) / [Chapter 4: Algorithmic Information and Machine Le...](#) / [Lesson 1: The Induction Pro...](#)



< Previous	 		Next >	Discussions
An illustration: Analogy making  Bookmark this page				All posts sorted by recent activity

Question: Can we say that Analogy is like building bridges between neighbouring islands?

Answer: 😊

Analogy is a form of reasoning and mental representation that works by connecting similar situations and transferring knowledge between them. This way of reasoning is commonly found in IQ tests, which means that it is thought to be central to human intelligence. It is also manifest in language use, as when we draw comparisons or speak in metaphors.

Solving analogies: Complexity can help!

CREDITS

UNDERSTANDING A THROUGH ALGORN THEORY
"SOLVING THE MYSTER

Content and presentation
Language coaching: Be
Video and illustrations:
Production: Institut Min

4:38 / 4:38

2.0x

consists of finding a transformation that would apply to both words. There exists infinitely many such transformation rules.
For instance:
(Rule 1) Take your word and add 'orang' at the end.
(Rule 2) Return

ty: $\mathcal{A}(A,B,A,B)$
etry: $\mathcal{A}(A,B,C,D)$
 $D,A,B)$
l
itation: $\mathcal{A}(A,B,C,D)$
 $C,B,D)$

n explains
roperties
ld.

Question: Can we process analogies automatically, knowing that complexity is not computable?

Answer: Yes, and we show it in the case of morphological analogies.

When you learn a new language, you often infer grammar rules long before discovering their explicit form in a grammar book. This process is typical of analogical reasoning. Let's say you are studying Finnish and that you learn the verb *puhua*. You discover that the first person in the present tense of this verb is *puhun*. Now, you want to conjugate the verb

nauraa in the same tense and in the same person. What do you suggest? The similarity of the structure lets you think that the answer is probably *nauran*. Congratulations! You are now able to conjugate the Finnish first group of verbs in the first person of the present tense! You can do it because you intuitively know how to process **morphological analogies**.

function with
rosa
the second half

function with
vita

ng ogical

Computing morphological analogies

Let's show that we can solve analogies on words using a minimal complexity principle. To do so, we consider a specialized programming language.

- A program in this language corresponds to a sequence of comma-delimited instructions.
- A sequence describes a series of concatenations. For instance, the program 'ab', 'cd' outputs the string abcd.
- The main characteristic of our language is the implementation of a memory as a push-down stack. At any time, an element can be stored into memory, using two let instructions (an opening one and a closing one). For instance, let, 'abc', let stores the string abc in memory.
- The operator mem is used to retrieve the last elements stored in memory. The i^{th} last element stored in memory can be retrieved with the instruction mem, i. By convention, the index starts with 0 (which means that mem, 0 retrieves the very last element saved in memory).

Code for strings 1

1/1 point (ungraded)

Which would be the output of the program

let, 'ab', let, let, 'cd', let, mem, 1 ?

- ☒ ab
- ☐ cd
- ☐ abcd
- ☐ abcdcd



Answer

Correct:

The program outputs ab . Values stored in memory are not considered for the output. Only the element at position 1 in memory is retrieved and displayed.

Submit

You have used 1 of 1 attempt

i Answers are displayed within the problem

- In addition, we allow the program to store operations in memory, in which case the parameters are denoted by ?n, where n is the index of the parameter. For instance, the instruction `let, ?0, ?1, let` stores an operation that concatenates its two operands.

Code for strings 2

1/1 point (ungraded)

Which of the following programs defines an operation which duplicates a string and introduces a hyphen to separate the duplicates?

☐ `let, ?0, '-', ?1, let`

☐ `let, '-', let`

☐ `let, ?0, ?0, let`

☒ `let, ?0, '-', ?0, let`

✓

Answer
Correct:
The correct program is
`let, ?0, '-', ?0, let` .

Submit

You have used 1 of 1 attempt

i Answers are displayed within the problem

A compiler for this language is given in the python script `analogy.py` . The compilation is called by the method `generate_string`, which takes as argument the instruction as a string. For instance:

```
$ python
>>> import analogy
>>> analogy.generate_string("abc,
ijk")
>>> 'abcijk'
>>> analogy.generate_string("let,
?0, -, ?0, let, mem, 0, orang")
>>> 'orang-orang'
>>>
analogy.instruction_complexity("l
et, ?0, -, ?0, let, mem, 0,
orang")
```

>>> 59

How can such a program be used to evaluate good analogies?
A solution consists of assuming that the quality of an analogy $A:B::C:D$ is measured by the complexity of the string 'A:B::C:D'.
The program estimates this complexity through the function `instruction_complexity(instruction_string)` which takes as argument the instruction as a string.

String generation 1

0/1 point (ungraded)

Select all programs that generate the string: `rosa:rosam`.

- ☒ `let, ?0, :, ?0, ?1, let, mem, 0, rosa, m`
✓
- ☐ `let, ?1, :, ?0, let, mem, 0, m, rosa`
- ☒ `let, ?0, a, :, ?0, a, m, let, mem, 0, ros`
✓
- ☒ `let, ?0, :, ?0, m, let, mem, 0, rosa`
✓
- ☐ `rosa:rosam`
✓



Explication
All answers are correct, but the second one.
Its output would be `rosa:m`

You have used 1 of 1 attempt

i Answers are displayed within the problem

String generation 2

1/1 point (ungraded)

Based on the estimates of `analogy.py`, how complex is the less complex among these programs?

✓

Answer: 59

Answer
Correct:
The complexity of `'let, ?0, :, ?0, m, let, mem, 0, rosa'` is 59. You'll notice that the complexity of the trivial instruction `'rosa:rosam'` is significantly larger, equal to 70.

Submit

You have
used 1 of 1
attempt

i Answers are displayed within
the problem

Solving analogies in this way is very time-consuming for two reasons. First, it requires the exploration of all possible solutions. Second, we have to explore all possible programs that generate each solution. A simplified solver is proposed in the function `restricted_search`. To execute it, just provide the desired analogy. The method will suggest a result based on complexity minimization. For instance, you can check the analogy *rosa:rosam::vita:vitam* by calling `restricted_search("rosa:rosam::vita:vitam")`.

```
$ python
>>> import analogy
>>>
analogy.restricted_search('rosa:rosam::vita:vitam')
Best instruction: let,?0,a,::,?
0,a,m,let,mem,0,ros,::,mem,0,vit
Expected result: vitam
Obtained result: vitam
```

Be careful: though the exploration space is restricted, the exploration can still take some time, especially if you ask for difficult analogies!

Analogy in Finnish

1/1 point (ungraded)

Execute the code to solve an analogy in Finnish:

`puhua:puhun::katsoa:katson` . Which instruction do you get as a result?

`let,?0,a,::,?0,n,let,mem,0,puhu,::,mem,0,katsc`



< Previous

Next >

CC BY-SA Some Rights Reserved



edX

[About](#)

[Affiliates](#)

[edX for Business](#)

[Open edX](#)

[Careers](#)

Legal

[Terms of Service & Honor Code](#)

[Privacy Policy](#)

[Accessibility Policy](#)

[Trademark Policy](#)

[Sitemap](#)

[Cookie Policy](#)

[Your Privacy Choices](#)

Connect

[Idea Hub](#)

[Contact Us](#)

[Help Center](#)

[Security](#)

[Media Kit](#)

